Efficient Attention for Async LLM Training Task

Abstract

Asynchronous decoding with shared context requires attention to preserve the historical visibility of each query. We describe an operator that groups queries by KV-block version, evaluates the groups with a FlashAttention-4 (FA4) variable-length kernel, and combines partial outputs using log-sum-exp (LSE). Query-relative rotary position embeddings (RoPE) allow keys to be rotated once per version. We give the forward and backward merge equations and report isolated forward-plus-backward measurements on NVIDIA H200 using a recorded ViZDoom rollout. The comparison uses a chunked FP32 PyTorch baseline and BF16 FA4 attention; each latency entry is a single measured sample.

FA4 supports custom masks, variable-length batches, and Hopper/Blackwell. Forward returns log-sum-exp (LSE); backward accepts its gradient. The pipeline is:

1. Group queries by KV-block version.
2. Evaluate the groups with an FA4 variable-length kernel.
3. Combine partial outputs with an LSE merge kernel.

1 Example

Generation order:

```
a0 a1 a2 b0 c0 c1 b1 b2 c2 c3
```

A is a shared prefix; B/C decode asynchronously (Hogwild) with mutual context access. Thus `c0/c1` see `b0`; `b1/b2` also see `c0/c1`.

Packed order: `[a0 a1 a2 | b0 b1 b2 | c0 c1 c2 c3]`. Figure 1 preserves execution-order causality, excluding future `b1/b2` from `c0/c1`.

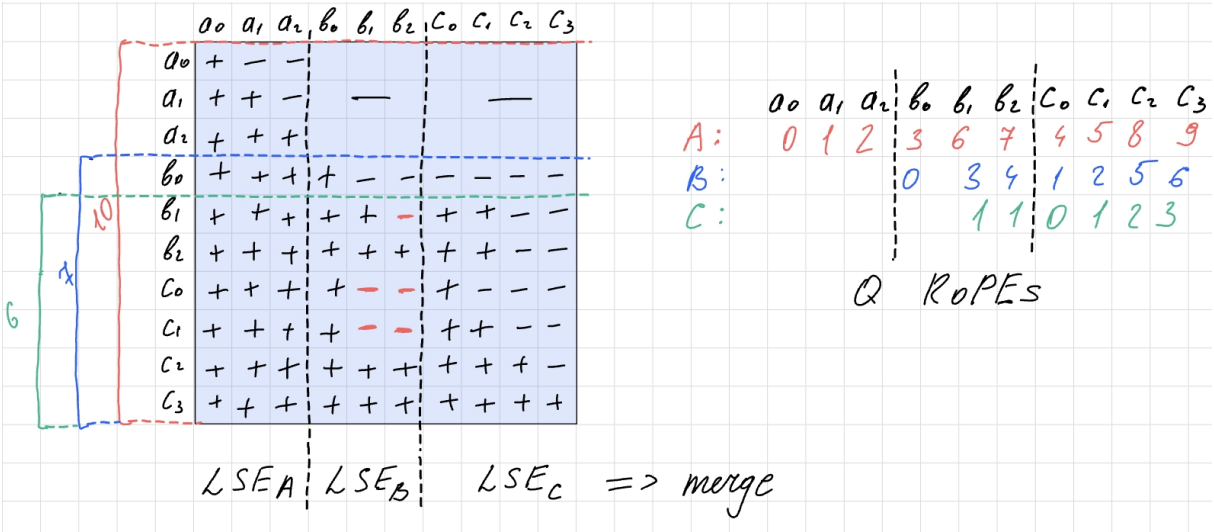


Figure 1. Historical visibility and query RoPE positions for `cache_view=[A,B,C]`.

2 KV blocks and versions

Block-local key positions: A/B use `[0,1,2]`; C uses `[0,1,2,3]`.

Prefill/decode extends the current version; each read records its historical [left, right) range.

Clear, `append_block`, and merge create versions. Prior reads retain their versions; inherited K/V may repeat.

3 Grouping and FA4 varlen

Queries are grouped by KV version; Q_A contains all readers of A.

KV group	Queries	Q / K count
A	`a0 a1 a2 b0 b1 b2 c0 c1 c2 c3`	10 / 3
B	`b0 b1 b2 c0 c1 c2 c3`	7 / 3
C	`b1 b2 c0 c1 c2 c3`	6 / 4

10 tokens yield 23 query copies with individual RoPE positions and masks. Referenced KV spans are packed once per version; repeated reads are preserved.

One varlen call evaluates the three groups with independent masks and softmax. Full score/weight matrices S/P are not materialized; only index plans are cached.

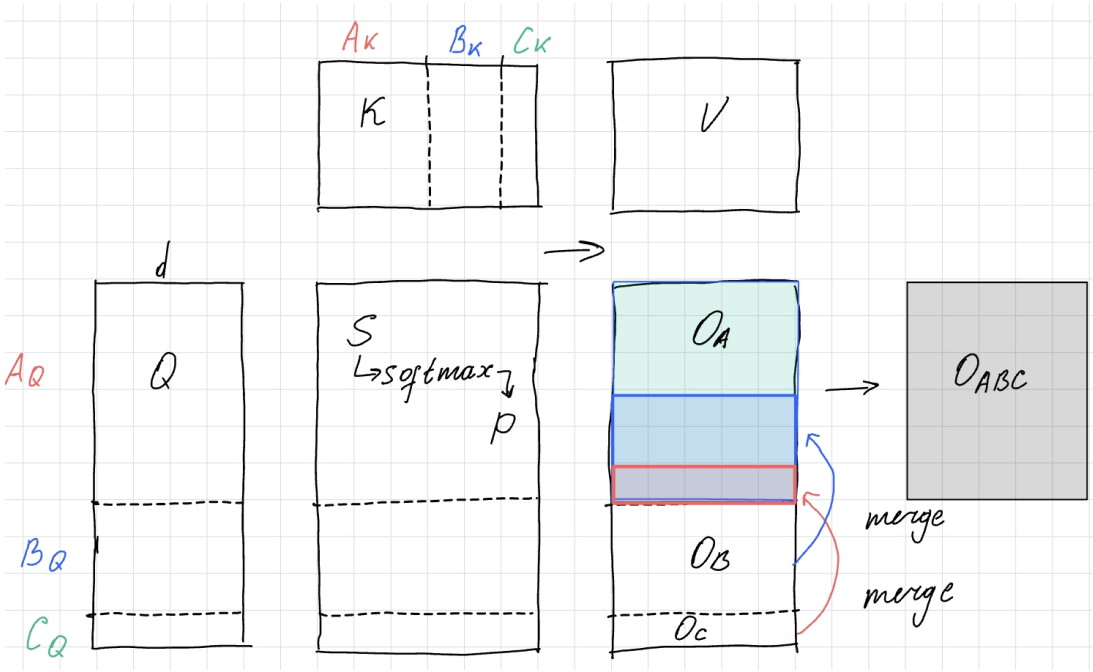


Figure 2. Query packing and partial-output merge (not to scale).

4 Query-relative RoPE

For query q, p denotes its position, s_g block g's offset in its cache view, and j the local key position:

$$\langle R(p - s_g)q, R(j)k\rangle = \langle R(p)q, R(s_g + j)k\rangle. \tag{1}$$

Keys are rotated once per version; query copies absorb view offsets.

For text decode in Figure 1, $p = |A| + |B| + |C| - 1$, using visible lengths after the current write. Block offsets are $s_A = 0$, $s_B = |A|$, $s_C = |A| + |B|$. mRoPE uses recorded 3D coordinates.

5 Differentiable LSE merge

FA4 returns an output and its softmax log-normalizer for each packed query row. One such row represents one recorded read `(version, left, right, position)`. An original query may have several reads, including repeated reads of the same version. The merge combines their outputs into one output per original query and head.

Let N be the number of original queries, $P = N_p$ the number of packed partial-query rows, H the number of query heads, and $D = d$ the head dimension. The tensor layouts are:

Tensor	Shape	Storage type
`parts`, `grad_parts`	$[P, H, D]$	input dtype
`lse`, `grad_lse`	$[H, P]$	FP32
`output`, `grad`	$[N, H, D]$	output dtype
`output_fp32`	$[N, H, D]$	FP32
`total_lse`	$[N, H]$	FP32
`indptr`, `indices`	$[N+1]$, $[P]$	int32

The input/output dtype is FP16, BF16, or FP32, and all LSE values use natural logarithms. A static CSR plan lists the partial rows belonging to original query n:

$$I_n = \texttt{indices}[\texttt{indptr}[n] : \texttt{indptr}[n+1]]. \tag{2}$$

Each partial row appears in exactly one I_n, exactly once. Repeated reads have distinct partial rows and remain distinct in this list.

For the following equations, fix one query n and head h. For $i \in I_n$, define

$$\begin{aligned} p_i &= \texttt{parts}[i, h, :] \in \mathbb{R}^D, & \ell_i &= \texttt{lse}[h, i] \in \mathbb{R}, \\ y &= \texttt{output_fp32}[n, h, :] \in \mathbb{R}^D, & T &= \texttt{total_lse}[n, h] \in \mathbb{R}. \end{aligned} \tag{3}$$

Here p_i is an already normalized partial attention output. The softmax in the merge acts on the scalar LSE values ℓ_i across partial rows; its coefficients mix the output vectors p_i.

5.1 Forward: mathematics and algorithm

For a group with at least one finite LSE, the merged log-normalizer, normalized coefficients, and output are

$$\begin{aligned} T &= \log \sum_{i \in I_n} e^{\ell_i}, & \alpha_i &= e^{\ell_i - T} \in \mathbb{R}, \\ y &= \sum_{i \in I_n} \alpha_i p_i = \frac{\sum_{i \in I_n} e^{\ell_i} p_i}{\sum_{i \in I_n} e^{\ell_i}} \in \mathbb{R}^D, & \sum_{i \in I_n} \alpha_i &= 1. \end{aligned} \tag{4}$$

The coefficients depend on `lse`; the values being combined come from `parts`. Their common normalization makes this output agree with attention over all recorded reads of the query.

The implementation evaluates Equation 4 with a fixed maximum and FP32 accumulators:

$$\begin{aligned} m &= \max_{i \in I_n} \ell_i, & \widetilde{w}_i &= e^{\ell_i - m}, \\ y &= \frac{\sum_{i \in I_n} \widetilde{w}_i p_i}{\sum_{i \in I_n} \widetilde{w}_i}, & T &= m + \log \sum_{i \in I_n} \widetilde{w}_i. \end{aligned} \tag{5}$$

Forward `weight` is the *unnormalized* $\widetilde{w}_i$. Algorithm 1 first finds m, then accumulates with fixed m and divides by `denominator` once. This two-pass implementation has no online rescaling and stores no weight tensor.

Algorithm 1 Two-pass LSE merge forward (`MergeForward.kernel`)

Require: `parts`$[P, H, D]$, `lse`$[H, P]$, CSR plan (`indptr`, `indices`)
Ensure: `output`$[N, H, D]$, saved `output_fp32`$[N, H, D]$, `total_lse`$[N, H]$

for all query/head pairs (n, h) in parallel, one warp per pair **do**
  $I_n \leftarrow$ `indices`[`indptr`$[n] : $ `indptr`$[n+1]]$
  maximum $\leftarrow -\infty$
  for all $i \in I_n$ **do** ▷ First pass
    maximum $\leftarrow \max($maximum, `lse`$[h, i])$
  end for
  accum $\leftarrow 0 \in \mathbb{R}^D$, denominator $\leftarrow 0$ ▷ FP32
  if maximum $\neq -\infty$ **then**
    for all $i \in I_n$ **do** ▷ Second pass
      weight $\leftarrow \exp($`lse`$[h, i] -$ maximum$)$
      denominator $\leftarrow$ denominator + weight
      accum $\leftarrow$ accum + weight $\cdot$ FP32(`parts`$[i, h, :]$)
    end for
  end if
  $y \leftarrow 0 \in \mathbb{R}^D$, $T \leftarrow -\infty$
  if denominator > 0 **then**
    $y \leftarrow$ accum/denominator
    $T \leftarrow$ maximum + log(denominator)
  end if
  `output_fp32`$[n, h, :] \leftarrow y$; lane 0 stores `total_lse`$[n, h] \leftarrow T$
  `output`$[n, h, :] \leftarrow \text{cast}_{\text{input}}(y)$
end for

For a fully masked group, all $\ell_i = -\infty$. The explicit convention is $y = 0$ and $T = -\infty$; the guarded second pass avoids evaluating $-\infty - (-\infty)$. The FP32 output is saved before the returned output is cast to the input dtype. Backward also retains `parts`, `lse`, and the CSR metadata.

Both kernels launch four warps per block and assign one warp to each query/head pair. Lane $\lambda \in \{0, \ldots, 31\}$ handles coordinates $d = \lambda + 32k < D$. In forward, each lane computes the same scalar maximum and denominator, while the vector accumulator is distributed across lanes. Lane 0 writes the shared scalar T.

5.2 Backward: mathematics and algorithm

Let $\mathcal{L}$ be a scalar loss and let the incoming gradient be

$$g = \frac{\partial \mathcal{L}}{\partial y} = \texttt{grad}[n, h, :] \in \mathbb{R}^D. \tag{6}$$

At the merge boundary, p_i and ℓ_i are separate inputs. A partial derivative with respect to one holds the other fixed. Backward reads the saved y and T, then recomputes the *normalized* coefficient

$$\alpha_i = \exp(\ell_i - T). \tag{7}$$

This is `weight` in `MergeBackward.kernel`; the output itself is taken from `output_fp32`.

For the derivative with respect to p_i, the LSE values and hence the coefficients are fixed. Each coordinate of p_i affects only the same coordinate of y, giving the diagonal Jacobian

$$\frac{\partial y}{\partial p_i} = \alpha_i I_D \in \mathbb{R}^{D \times D}, \qquad \boxed{\frac{\partial \mathcal{L}}{\partial p_i} = \alpha_i g \in \mathbb{R}^D}. \tag{8}$$

Here I_D is the $D \times D$ identity matrix.

For the derivative with respect to ℓ_i, hold all p_j fixed and differentiate the numerator and denominator of Equation 4. Only their ith terms depend on ℓ_i. Applying the quotient rule coordinate by coordinate yields

$$\begin{aligned} \frac{\partial y}{\partial \ell_i} &= \frac{e^{\ell_i} p_i}{\sum_{j \in I_n} e^{\ell_j}} - \frac{e^{\ell_i} \sum_{j \in I_n} e^{\ell_j} p_j}{\left(\sum_{j \in I_n} e^{\ell_j}\right)^2} \\ &= \frac{e^{\ell_i}}{\sum_{j \in I_n} e^{\ell_j}} \left(p_i - \frac{\sum_{j \in I_n} e^{\ell_j} p_j}{\sum_{j \in I_n} e^{\ell_j}} \right) \\ &= \alpha_i (p_i - y) \in \mathbb{R}^D. \end{aligned} \tag{9}$$

The $-y$ term comes from differentiating the denominator. Although T is saved for numerical evaluation in backward, mathematically it depends on every ℓ_i; that dependence is already included here.

One scalar ℓ_i influences the loss through all D coordinates of y. The multivariate chain rule sums those scalar contributions:

$$\begin{aligned} \frac{\partial \mathcal{L}}{\partial \ell_i} &= \sum_{d=0}^{D-1} \frac{\partial \mathcal{L}}{\partial y_d} \frac{\partial y_d}{\partial \ell_i} \\ &= \alpha_i \sum_{d=0}^{D-1} g_d (p_{i,d} - y_d) = \boxed{\alpha_i \langle g, p_i - y \rangle \in \mathbb{R}}. \end{aligned} \tag{10}$$

Thus `grad_parts` is a vector per partial/head, whereas `grad_lse` is one scalar per partial/head. The dot product is the coordinate sum required by the chain rule.

Algorithm 2 LSE merge backward with coefficient recomputation (`MergeBackward.kernel`)

Require: Saved `parts`, `lse`, CSR plan, `output_fp32`, `total_lse`
Require: Incoming `grad`$[N, H, D]$
Ensure: `grad_parts`$[P, H, D]$, `grad_lse`$[H, P]$

for all query/head pairs (n, h) in parallel, one warp per pair **do**
  $I_n \leftarrow$ `indices`[`indptr`$[n] :$ `indptr`$[n+1]]$
  $y \leftarrow$ `output_fp32`$[n, h, :]$, $T \leftarrow$ `total_lse`$[n, h]$
  $g \leftarrow \mathrm{FP32}($`grad`$[n, h, :])$
  for all $i \in I_n$ **do**
    $\alpha_i \leftarrow 0$
    if $T \neq -\infty$ **then**
      $\alpha_i \leftarrow \exp($`lse`$[h, i] - T)$ ▷ Recompute normalized weight
    end if
    for all warp lanes $\lambda = 0, \ldots, 31$ in parallel **do**
      $\mathrm{dot}_\lambda \leftarrow 0$ ▷ FP32 lane-local accumulator
      for $k = 0, \ldots, \lceil D/32 \rceil - 1$ **do**
        $d \leftarrow \lambda + 32k$
        if $d < D$ **then**
          $\mathrm{difference} \leftarrow \mathrm{FP32}($`parts`$[i, h, d]) - y_d$
          $\mathrm{dot}_\lambda \leftarrow \mathrm{dot}_\lambda + g_d \cdot \mathrm{difference}$
          `grad_parts`$[i, h, d] \leftarrow \mathrm{cast}_{\mathrm{input}}(\alpha_i g_d)$
        end if
      end for
    end for
    $\mathrm{dot} \leftarrow \mathrm{WarpReduceSum}(\mathrm{dot}_\lambda)$
    Lane 0 stores `grad_lse`$[h, i] \leftarrow \alpha_i \cdot \mathrm{dot}$
  end for
end for

The subtraction and dot accumulation use FP32, and the warp reduction combines the lane-local sums before lane 0 writes the scalar LSE gradient. For $T = -\infty$, the coefficient is explicitly set to zero, implementing the fully masked zero-gradient convention without an undefined exponential.

The CSR ownership invariant gives every `grad_parts` and `grad_lse` element a unique writer, so this merge needs no atomic updates. Both gradients flow into FA4 backward; the resulting Q/K/V gradients continue through RoPE and the packed gathers. Gather backward accumulates contributions when different reads refer to the same source row. CSR metadata is nondifferentiable, and this implementation supports first-order gradients.

6 Performance

6.1 Workload

The workload is one ViZDoom **Health Gathering** rollout collected with Qwen3.5-27B: 1,392 environment actions and 718,798 recorded tokens. An asynchronous descriptor and thinker provide context to an action-selection probe. The logical KV layout is:

```
Prompt | History | Diff | ImagePair(previous, current)
       | Descriptor | Thinker | Probe
```

ImagePair stores both frames in one KV block; role sub-blocks are abbreviated. Queries select different views of these blocks. Each action adds an observation/thought/action/outcome record to Diff (Delta in the trace). Every 100 actions, `append_block(History, Diff)` extends History, then Diff is cleared. Images and temporary role blocks are refreshed each round.

Benchmarks use initial prefixes ending at complete recorded batches. N counts recorded tokens; $C_{\max}$ is the maximum number of visible keys per query. The largest tested prefix has $N = 65{,}437$ and $C_{\max} = 6{,}533$.

6.2 Measurement protocol

We measure one isolated attention operator, forward plus backward (F+B), including packing, RoPE, attention, and merge. Model projections, other layers, loss, and optimizer updates are excluded. In the tables, **FA4** denotes this complete operator path.

Each backend uses 10 F+B warmups followed by one measured F+B on NVIDIA H200, with checkpointing disabled. Times are synchronized wall time; metadata construction, JIT, input generation, and gradient clearing are excluded. Backend pairs use identical synthetic BF16 Q/K/V and upstream gradients (seed 197), with recorded visibility and mRoPE. Geometries run on separate GPUs; paired backends run sequentially on the same GPU. OMP/MKL and Torch intra-op threads are set to 1.

The `torch_optimized` baseline shares K/V preparation across 128-query chunks and computes attention in FP32 with TF32 disabled. FA4 uses BF16 attention with FP32 statistics and merge accumulation. Reported ratios therefore include arithmetic and implementation differences.

6.3 Isolated latency

Speedup is $T_{\text{Torch}}/T_{\text{FA4}}$. H and H_{KV} denote query and KV head counts; d is the head dimension. Model names specify attention geometry; no model weights are loaded.

Table 1. F+B wall time (ms), Qwen3.5-27B geometry: $H = 24$, $H_{\text{KV}} = 4$, $d = 256$.

N	$C_{\max}$	Torch optimized (ms)	FA4 (ms)	Speedup
1,956	805	54.37	3.33	16.35×
4,008	985	113.66	5.78	19.66×
8,144	1,361	241.28	11.96	20.17×
16,240	2,109	495.41	26.59	18.63×
32,758	3,602	1305.22	65.49	19.93×
65,437	6,533	3995.30	160.07	24.96×

Table 2. F+B wall time (ms), Qwen3.5-0.8B geometry: $H = 8$, $H_{\mathrm{KV}} = 2$, $d = 256$.

N	$C_{\max}$	Torch optimized (ms)	FA4 (ms)	Speedup
1,956	805	49.49	2.98	16.61×
4,008	985	109.32	3.17	34.50×
8,144	1,361	242.57	4.76	50.95×
16,240	2,109	506.62	9.43	53.70×
32,758	3,602	995.85	22.87	43.55×
65,437	6,533	2653.59	55.77	47.58×

All isolated cases completed without OOM. Each entry is one measured sample; no confidence interval is estimated. Ratios use unrounded times. Results apply to this trace's visibility structure, not arbitrary workloads of equal N.

6.4 GPU-time breakdown

A separate Nsight Systems capture measures the FA4 path for the 27B geometry at $N = 65{,}437$, after 10 warmups, with checkpointing disabled. This prefix contains 1,014 KV versions, $N_p = 363{,}714$ query copies, and 70,098 packed KV rows. The capture includes one complete F+B and the `dLSE` path.

Table 3. Sum of GPU kernel durations (ms). Shares are normalized by the total kernel duration, excluding memory operations, CPU time, and timeline gaps.

Component	Forward (ms)	Backward (ms)	Total (ms)	Share (%)
FA4	17.85	89.42	107.27	66.75
LSE merge	2.37	6.04	8.41	5.23
Packing / RoPE / other	17.45	27.56	45.01	28.01
Total	37.67	123.03	160.70	100.00

CUDA launch correlation and NVTX ranges assign all 152 kernels exactly once. The FA4 category includes its preparation and gradient-processing kernels. Memcpy/memset account for a separate 0.013 ms. Totals and shares use unrounded durations.

Profiled wall time is 165.43 ms, including tracing overhead; Table 1 uses the independent, unprofiled 160.07 ms measurement. The merge contribution is 5.23% of summed GPU kernel time for this workload.

Runtime: PyTorch 2.13.0+cu130, CUDA 13.0, FA4 4.0.0b37, CUTLASS DSL 4.6.2. Timing and profiling runs use matching FA4 backend, plan, mask, and merge sources.